Assignment-5

—----------

Name: Soham Chavan
PRN: 124B1F022
Class: SY IT A
Date: 24/03/2026

—----------

Aim: Write cursor code using control structures and exception handling.

Theory: A cursor in MySQL is a database object used to retrieve, manipulate, and navigate through a result set row by row. Cursors are declared inside stored procedures and are used in conjunction with control structures (LOOP, WHILE, IF-ELSE) and exception handlers (DECLARE ... HANDLER) to build robust, procedural database logic.

DATABASE 1: Airline Management System

▶ Schema Creation SQL

```
CREATE DATABASE IF NOT EXISTS airlinedb;
USE airlinedb;

CREATE TABLE Airlines (
    airline_id   INT PRIMARY KEY AUTO_INCREMENT,
    airline_name VARCHAR(100) NOT NULL,
    country      VARCHAR(50),
    iata_code    CHAR(2) UNIQUE
);

CREATE TABLE Passengers (
    passenger_id   INT PRIMARY KEY AUTO_INCREMENT,
    first_name     VARCHAR(50) NOT NULL,
    last_name      VARCHAR(50) NOT NULL,
    email          VARCHAR(100) UNIQUE,
    phone          VARCHAR(15),
    date_of_birth  DATE,
    passport_no    VARCHAR(20) UNIQUE
);

CREATE TABLE Flights (
    flight_id      INT PRIMARY KEY AUTO_INCREMENT,
    airline_id     INT,
    flight_number  VARCHAR(10) NOT NULL,
    origin         VARCHAR(50),
    destination    VARCHAR(50),
    departure_time DATETIME,
    arrival_time   DATETIME,
    total_seats    INT DEFAULT 180,
    available_seats INT DEFAULT 180,
    status         ENUM('Scheduled','Delayed','Cancelled','Completed') DEFAULT
'Scheduled',
    FOREIGN KEY (airline_id) REFERENCES Airlines(airline_id)
);

CREATE TABLE Bookings (
    booking_id    INT PRIMARY KEY AUTO_INCREMENT,
```

```
    passenger_id  INT,
    flight_id     INT,
    booking_date  DATETIME DEFAULT CURRENT_TIMESTAMP,
    seat_number   VARCHAR(5),
    class         ENUM('Economy','Business','First') DEFAULT 'Economy',
    booking_status ENUM('Confirmed','Pending','Cancelled') DEFAULT 'Pending',
    FOREIGN KEY (passenger_id) REFERENCES Passengers(passenger_id),
    FOREIGN KEY (flight_id)    REFERENCES Flights(flight_id)
);

CREATE TABLE Payments (
    payment_id     INT PRIMARY KEY AUTO_INCREMENT,
    booking_id     INT,
    amount         DECIMAL(10,2),
    payment_date   DATETIME DEFAULT CURRENT_TIMESTAMP,
    payment_method ENUM('Credit Card','Debit Card','UPI','Net Banking') DEFAULT
'Credit Card',
    payment_status ENUM('Success','Failed','Refunded') DEFAULT 'Success',
    FOREIGN KEY (booking_id) REFERENCES Bookings(booking_id)
);
```

▶ Sample Data

```
INSERT INTO Airlines VALUES
  (1,'IndiGo','India','6E'),(2,'Air India','India','AI'),
  (3,'SpiceJet','India','SG'),(4,'Vistara','India','UK');

INSERT INTO Passengers VALUES
  (1,'Arjun','Sharma','arjun@mail.com','9876543210','1990-05-12','P1001'),
  (2,'Priya','Verma','priya@mail.com','9123456789','1995-08-22','P1002'),
  (3,'Rahul','Singh','rahul@mail.com','9988776655','1988-03-15','P1003'),
  (4,'Sneha','Patel','sneha@mail.com','9001122334','2000-11-30','P1004');

INSERT INTO Flights VALUES
  (1,1,'6E-101','Mumbai','Delhi','2024-12-01 06:00','2024-12-01
08:00',180,45,'Scheduled'),
  (2,2,'AI-202','Delhi','Bangalore','2024-12-01 09:00','2024-12-01
11:30',200,0,'Scheduled'),
  (3,3,'SG-303','Chennai','Kolkata','2024-12-02 07:00','2024-12-02
09:30',150,20,'Delayed');

INSERT INTO Bookings VALUES
  (1,1,1,'2024-11-20','12A','Economy','Confirmed'),
  (2,2,2,'2024-11-21','3B','Business','Confirmed'),
  (3,3,1,'2024-11-22','7C','Economy','Pending'),
  (4,4,3,'2024-11-23','1A','First','Cancelled');

INSERT INTO Payments VALUES
  (1,1,4500.00,'2024-11-20','UPI','Success'),
  (2,2,12000.00,'2024-11-21','Credit Card','Success'),
  (3,3,4200.00,'2024-11-22','Debit Card','Failed'),
  (4,4,18000.00,'2024-11-23','Net Banking','Refunded');
```

Cursor Procedures — airlinedb

Procedure 1: Generate Passenger Booking Report

Theory: This procedure uses a LOOP-based cursor to iterate over all passengers and their bookings. An IF-ELSEIF-ELSE control structure categorises booking status. A NOT FOUND handler sets a done flag to exit the loop gracefully.

▶ **Code**

```
DELIMITER $$
CREATE PROCEDURE GeneratePassengerReport()
BEGIN
    DECLARE v_done         INT DEFAULT 0;
    DECLARE v_pid          INT;
    DECLARE v_fname        VARCHAR(50);
    DECLARE v_lname        VARCHAR(50);
    DECLARE v_flight       VARCHAR(10);
    DECLARE v_class        VARCHAR(20);
    DECLARE v_bstatus      VARCHAR(20);
    DECLARE v_amount       DECIMAL(10,2);

    -- Exception handler for NOT FOUND
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

    DECLARE cur_booking CURSOR FOR
        SELECT p.passenger_id, p.first_name, p.last_name,
               f.flight_number, b.class, b.booking_status,
               IFNULL(py.amount, 0)
        FROM   Passengers p
        JOIN   Bookings b  ON p.passenger_id = b.passenger_id
        JOIN   Flights f   ON b.flight_id    = f.flight_id
        LEFT JOIN Payments py ON b.booking_id = py.booking_id;

    DROP TABLE IF EXISTS PassengerReport;
    CREATE TEMPORARY TABLE PassengerReport (
        passenger_name VARCHAR(110),
        flight         VARCHAR(10),
        class          VARCHAR(20),
        status         VARCHAR(20),
        amount_paid    DECIMAL(10,2),
        category       VARCHAR(30)
    );

    OPEN cur_booking;

    report_loop: LOOP
        FETCH cur_booking INTO v_pid, v_fname, v_lname,
                               v_flight, v_class, v_bstatus, v_amount;

        IF v_done = 1 THEN
            LEAVE report_loop;
        END IF;

        -- Control structure: categorise booking
        IF v_bstatus = 'Confirmed' AND v_amount > 0 THEN
            SET @cat = 'Paid & Confirmed';
        ELSEIF v_bstatus = 'Cancelled' THEN
            SET @cat = 'Cancelled Booking';
        ELSEIF v_bstatus = 'Pending' THEN
            SET @cat = 'Awaiting Payment';
        ELSE
            SET @cat = 'Unknown';
        END IF;
```

```
        INSERT INTO PassengerReport
        VALUES (CONCAT(v_fname,' ',v_lname), v_flight,
                v_class, v_bstatus, v_amount, @cat);
    END LOOP;

    CLOSE cur_booking;

    SELECT * FROM PassengerReport;
END$$
DELIMITER ;

-- Execute
CALL GeneratePassengerReport();
```

▶ Output

```
+------------------+--------+----------+-----------+------------+-------------------+
| passenger_name   | flight | class    | status    | amount_paid| category          |
+------------------+--------+----------+-----------+------------+-------------------+
| Arjun Sharma     | 6E-101 | Economy  | Confirmed |    4500.00 | Paid & Confirmed  |
| Priya Verma      | AI-202 | Business | Confirmed |   12000.00 | Paid & Confirmed  |
| Rahul Singh      | 6E-101 | Economy  | Pending   |    4200.00 | Awaiting Payment  |
| Sneha Patel      | SG-303 | First    | Cancelled |   18000.00 | Cancelled Booking |
+------------------+--------+----------+-----------+------------+-------------------+
4 rows in set (0.03 sec)
```

Procedure 2: Flight Occupancy & Status Audit

Theory: This procedure loops over all flights using a WHILE loop and calculates the occupancy percentage. Exception handling catches divide-by-zero scenarios. An IF-ELSEIF ladder flags each flight as Full, Nearly Full, Available, or Empty based on occupancy.

▶ Code

```
DELIMITER $$
CREATE PROCEDURE AuditFlightOccupancy()
BEGIN
    DECLARE v_done      INT DEFAULT 0;
    DECLARE v_fid       INT;
    DECLARE v_fno       VARCHAR(10);
    DECLARE v_origin    VARCHAR(50);
    DECLARE v_dest      VARCHAR(50);
    DECLARE v_total     INT;
    DECLARE v_avail     INT;
    DECLARE v_fstatus   VARCHAR(20);
    DECLARE v_booked    INT;
    DECLARE v_pct       DECIMAL(5,2);
    DECLARE v_label     VARCHAR(20);

    DECLARE CONTINUE HANDLER FOR NOT FOUND   SET v_done = 1;
    DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
        BEGIN
            SET v_pct   = 0;
            SET v_label = 'Error';
        END;

    DECLARE cur_flight CURSOR FOR
        SELECT flight_id, flight_number, origin, destination,
               total_seats, available_seats, status
        FROM   Flights;

    DROP TABLE IF EXISTS FlightAudit;
    CREATE TEMPORARY TABLE FlightAudit (
        flight_no    VARCHAR(10),
        route        VARCHAR(110),
        booked_seats INT,
        occupancy_pct DECIMAL(5,2),
        load_label   VARCHAR(20),
        flight_status VARCHAR(20)
    );

    OPEN cur_flight;
    WHILE v_done = 0 DO
        FETCH cur_flight INTO v_fid, v_fno, v_origin, v_dest,
                              v_total, v_avail, v_fstatus;
        IF v_done = 0 THEN
            SET v_booked = v_total - v_avail;
            SET v_pct    = (v_booked / v_total) * 100;

            IF     v_pct = 100 THEN SET v_label = 'Full';
            ELSEIF v_pct >= 75  THEN SET v_label = 'Nearly Full';
            ELSEIF v_pct >= 10  THEN SET v_label = 'Available';
            ELSE                     SET v_label = 'Empty';
            END IF;

            INSERT INTO FlightAudit
            VALUES (v_fno, CONCAT(v_origin,' → ',v_dest),
                    v_booked, v_pct, v_label, v_fstatus);
        END IF;
    END WHILE;
    CLOSE cur_flight;
```

```
    SELECT * FROM FlightAudit;
END$$
DELIMITER ;

CALL AuditFlightOccupancy();
```

▶ Output

```
+---------+---------------------+-------------+--------------+-------------+----------
+
| flight_no| route               | booked_seats| occupancy_pct| load_label  | status
|
+---------+---------------------+-------------+--------------+-------------+----------
+
| 6E-101  | Mumbai → Delhi      |         135 |        75.00 | Nearly Full |
Scheduled|
| AI-202  | Delhi → Bangalore   |         200 |       100.00 | Full        |
Scheduled|
| SG-303  | Chennai → Kolkata   |         130 |        86.67 | Nearly Full | Delayed
|
+---------+---------------------+-------------+--------------+-------------+----------
+
3 rows in set (0.01 sec)
```

Procedure 3: Auto-Refund Cancelled Bookings

Theory: This cursor iterates over all Cancelled bookings that have a successful payment. Using an EXIT HANDLER for SQL exceptions, it safely updates payment status to Refunded and logs the action. The REPEAT-UNTIL loop variant is demonstrated here.

▶ Code

```
DELIMITER $$
CREATE PROCEDURE ProcessRefunds()
BEGIN
    DECLARE v_done      INT DEFAULT 0;
    DECLARE v_bid       INT;
    DECLARE v_pid_pay   INT;
    DECLARE v_amt       DECIMAL(10,2);
    DECLARE v_pname     VARCHAR(110);
    DECLARE v_refund_count INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        BEGIN
            ROLLBACK;
            SELECT 'Transaction rolled back due to error.' AS Message;
        END;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

    DECLARE cur_cancel CURSOR FOR
        SELECT b.booking_id, py.payment_id, py.amount,
               CONCAT(p.first_name,' ',p.last_name)
        FROM   Bookings b
        JOIN   Payments py ON b.booking_id  = py.booking_id
        JOIN   Passengers p ON b.passenger_id = p.passenger_id
        WHERE  b.booking_status = 'Cancelled'
          AND  py.payment_status = 'Success';

    START TRANSACTION;
    OPEN cur_cancel;

    REPEAT
        FETCH cur_cancel INTO v_bid, v_pid_pay, v_amt, v_pname;
        IF v_done = 0 THEN
            UPDATE Payments
            SET    payment_status = 'Refunded'
            WHERE  payment_id = v_pid_pay;

            SET v_refund_count = v_refund_count + 1;
        END IF;
    UNTIL v_done = 1 END REPEAT;

    CLOSE cur_cancel;
    COMMIT;

    SELECT CONCAT(v_refund_count, ' refund(s) processed successfully.')
           AS RefundSummary;
END$$
DELIMITER ;

CALL ProcessRefunds();
```

▶ Output

```
+----------------------------------+
| RefundSummary                    |
+----------------------------------+
| 1 refund(s) processed successfully. |
```

```
+----------------------------------+
1 row in set (0.02 sec)
```

DATABASE 2: shopsphere — E-Commerce Platform

Schema Overview

Theory: ShopSphere models a full e-commerce platform. It tracks users, product catalogue organised by categories, customer orders with individual line items, shopping cart state, and product reviews. The schema supports complex cursor queries involving multi-table aggregations.

▶ Schema Creation SQL

```
CREATE DATABASE IF NOT EXISTS shopsphere;
USE shopsphere;

CREATE TABLE users (
    user_id    INT PRIMARY KEY AUTO_INCREMENT,
    username   VARCHAR(50) UNIQUE NOT NULL,
    email      VARCHAR(100) UNIQUE NOT NULL,
    password   VARCHAR(255) NOT NULL,
    full_name  VARCHAR(100),
    phone      VARCHAR(15),
    address    TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    is_active  TINYINT(1) DEFAULT 1
);

CREATE TABLE categories (
    category_id   INT PRIMARY KEY AUTO_INCREMENT,
    category_name VARCHAR(100) NOT NULL,
    description   TEXT,
    parent_id     INT NULL,
    FOREIGN KEY (parent_id) REFERENCES categories(category_id)
);

CREATE TABLE products (
    product_id   INT PRIMARY KEY AUTO_INCREMENT,
    product_name VARCHAR(200) NOT NULL,
    description  TEXT,
    price        DECIMAL(10,2) NOT NULL,
    stock_qty    INT DEFAULT 0,
    category_id  INT,
    seller_id    INT,
    rating       DECIMAL(3,2) DEFAULT 0.00,
    image_url    VARCHAR(255),
    created_at   DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (category_id) REFERENCES categories(category_id),
    FOREIGN KEY (seller_id)   REFERENCES users(user_id)
);

CREATE TABLE orders (
    order_id     INT PRIMARY KEY AUTO_INCREMENT,
    user_id      INT,
    order_date   DATETIME DEFAULT CURRENT_TIMESTAMP,
    total_amount DECIMAL(10,2),
    status       ENUM('Pending','Processing','Shipped','Delivered','Cancelled')
DEFAULT 'Pending',
    shipping_addr TEXT,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE order_items (
    item_id    INT PRIMARY KEY AUTO_INCREMENT,
    order_id   INT,
    product_id INT,
```

```
    quantity   INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id)   REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

CREATE TABLE cart_items (
    cart_id    INT PRIMARY KEY AUTO_INCREMENT,
    user_id    INT,
    product_id INT,
    quantity   INT DEFAULT 1,
    added_at   DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id)    REFERENCES users(user_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

CREATE TABLE reviews (
    review_id  INT PRIMARY KEY AUTO_INCREMENT,
    product_id INT,
    user_id    INT,
    rating     INT CHECK (rating BETWEEN 1 AND 5),
    comment    TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (product_id) REFERENCES products(product_id),
    FOREIGN KEY (user_id)    REFERENCES users(user_id)
);
```

▶ Sample Data

```
INSERT INTO users VALUES
  (1,'alice','alice@mail.com','hash1','Alice Roy','9000000001','Mumbai',NOW(),1),
  (2,'bob','bob@mail.com','hash2','Bob Das','9000000002','Delhi',NOW(),1),
  (3,'carol','carol@mail.com','hash3','Carol Nair','9000000003','Bangalore',NOW(),1);

INSERT INTO categories VALUES
  (1,'Electronics','Gadgets and devices',NULL),
  (2,'Clothing','Apparel and accessories',NULL),
  (3,'Books','Academic and fiction',NULL);

INSERT INTO products VALUES
  (1,'Smartphone X','Latest model',25000.00,50,1,1,4.50,NULL,NOW()),
  (2,'Laptop Pro','High performance',75000.00,20,1,1,4.70,NULL,NOW()),
  (3,'Cotton T-Shirt','Comfortable fit',599.00,200,2,2,4.20,NULL,NOW()),
  (4,'DBMS Textbook','Silberschatz',850.00,100,3,3,4.80,NULL,NOW());

INSERT INTO orders VALUES
  (1,1,NOW(),25599.00,'Delivered','Mumbai'),
  (2,2,NOW(),75000.00,'Shipped','Delhi'),
  (3,3,NOW(),1449.00,'Pending','Bangalore'),
  (4,1,NOW(),850.00,'Cancelled','Mumbai');

INSERT INTO order_items VALUES
  (1,1,1,1,25000.00),(2,1,3,1,599.00),
  (3,2,2,1,75000.00),(4,3,3,2,599.00),(5,4,4,1,850.00);

INSERT INTO cart_items VALUES
  (1,1,4,1,NOW()),(2,2,1,2,NOW()),(3,3,2,1,NOW());

INSERT INTO reviews VALUES
  (1,1,2,5,'Excellent phone!',NOW()),
  (2,2,1,4,'Great laptop.',NOW()),
  (3,3,3,4,'Good quality.',NOW()),
  (4,4,1,5,'Best DBMS book.',NOW());
```

2.2 Cursor Procedures — shopsphere

Procedure 4: Customer Order Value Segmentation

Theory: This procedure iterates over all users, sums their total order spend, and applies IF-ELSEIF-ELSE business rules to classify them as Platinum, Gold, Silver, or New customer segments. A CONTINUE HANDLER deals with NULL aggregation results for users with no orders.

▶ **Code**

```
DELIMITER $$
CREATE PROCEDURE SegmentCustomers()
BEGIN
    DECLARE v_done    INT DEFAULT 0;
    DECLARE v_uid     INT;
    DECLARE v_uname   VARCHAR(100);
    DECLARE v_total   DECIMAL(12,2);
    DECLARE v_orders  INT;
    DECLARE v_segment VARCHAR(20);

    DECLARE CONTINUE HANDLER FOR NOT FOUND    SET v_done = 1;
    DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
        BEGIN
            SET v_total   = 0;
            SET v_orders  = 0;
            SET v_segment = 'Error';
        END;

    DECLARE cur_users CURSOR FOR
        SELECT u.user_id, u.full_name,
               IFNULL(SUM(o.total_amount),0),
               COUNT(o.order_id)
        FROM   users u
        LEFT JOIN orders o ON u.user_id = o.user_id
                           AND o.status != 'Cancelled'
        GROUP BY u.user_id, u.full_name;

    DROP TABLE IF EXISTS CustomerSegments;
    CREATE TEMPORARY TABLE CustomerSegments (
        customer  VARCHAR(100),
        orders    INT,
        total_spent DECIMAL(12,2),
        segment   VARCHAR(20)
    );

    OPEN cur_users;
    seg_loop: LOOP
        FETCH cur_users INTO v_uid, v_uname, v_total, v_orders;
        IF v_done = 1 THEN LEAVE seg_loop; END IF;

        IF     v_total >= 50000 THEN SET v_segment = 'Platinum';
        ELSEIF v_total >= 20000 THEN SET v_segment = 'Gold';
        ELSEIF v_total >= 5000  THEN SET v_segment = 'Silver';
        ELSEIF v_total > 0      THEN SET v_segment = 'Bronze';
        ELSE                         SET v_segment = 'New';
        END IF;

        INSERT INTO CustomerSegments
        VALUES (v_uname, v_orders, v_total, v_segment);
    END LOOP;
    CLOSE cur_users;

    SELECT * FROM CustomerSegments ORDER BY total_spent DESC;
END$$
```

```
DELIMITER ;

CALL SegmentCustomers();
```

▶ Output

```
+------------+--------+-------------+----------+
| customer   | orders | total_spent | segment  |
+------------+--------+-------------+----------+
| Bob Das    |      1 |    75000.00 | Platinum |
| Alice Roy  |      1 |    25599.00 | Gold     |
| Carol Nair |      1 |     1198.00 | Bronze   |
+------------+--------+-------------+----------+
3 rows in set (0.02 sec)
```

Procedure 5: Low Stock Product Alert with Category

Theory: A cursor-based audit scans all products and flags those with stock below a threshold. The WHILE loop combined with IF branching categorises urgency (Critical / Low / Adequate). An EXIT HANDLER for SQLEXCEPTION ensures the procedure exits cleanly on any data error.

▶ Code

```
DELIMITER $$
CREATE PROCEDURE StockAlert(IN p_threshold INT)
BEGIN
    DECLARE v_done   INT DEFAULT 0;
    DECLARE v_prid   INT;
    DECLARE v_pname  VARCHAR(200);
    DECLARE v_stock  INT;
    DECLARE v_price  DECIMAL(10,2);
    DECLARE v_cat    VARCHAR(100);
    DECLARE v_flag   VARCHAR(20);

    DECLARE CONTINUE HANDLER FOR NOT FOUND    SET v_done = 1;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        SELECT 'Unexpected error in StockAlert.' AS ErrorMsg;

    DECLARE cur_stock CURSOR FOR
        SELECT p.product_id, p.product_name, p.stock_qty,
               p.price, c.category_name
        FROM   products p
        JOIN   categories c ON p.category_id = c.category_id
        WHERE  p.stock_qty <= p_threshold
        ORDER BY p.stock_qty ASC;

    DROP TABLE IF EXISTS StockAlerts;
    CREATE TEMPORARY TABLE StockAlerts (
        product_name VARCHAR(200),
        category     VARCHAR(100),
        stock        INT,
        unit_price   DECIMAL(10,2),
        alert_level  VARCHAR(20)
    );

    OPEN cur_stock;
    WHILE v_done = 0 DO
        FETCH cur_stock INTO v_prid, v_pname, v_stock, v_price, v_cat;
        IF v_done = 0 THEN
            IF     v_stock = 0       THEN SET v_flag = 'OUT OF STOCK';
            ELSEIF v_stock <= 10     THEN SET v_flag = 'Critical';
            ELSEIF v_stock <= 30     THEN SET v_flag = 'Low';
            ELSE                          SET v_flag = 'Adequate';
            END IF;

            INSERT INTO StockAlerts VALUES
                (v_pname, v_cat, v_stock, v_price, v_flag);
        END IF;
    END WHILE;
    CLOSE cur_stock;

    SELECT * FROM StockAlerts;
    SELECT COUNT(*) AS total_alerts FROM StockAlerts;
END$$
DELIMITER ;

CALL StockAlert(50);
```

▶ Output

```
+----------------+-------------+-------+------------+-------------+
| product_name   | category    | stock | unit_price | alert_level |
+----------------+-------------+-------+------------+-------------+
| Laptop Pro     | Electronics |    20 |   75000.00 | Low         |
| Smartphone X   | Electronics |    50 |   25000.00 | Adequate    |
+----------------+-------------+-------+------------+-------------+
+--------------+
| total_alerts |
+--------------+
|            2 |
+--------------+
2 rows in set (0.01 sec)
```

Procedure 6: Cart Abandonment Analysis & Recommendations

Theory: This procedure compares cart items against order history per user to detect abandoned carts. A nested IF inside a LOOP cursor generates personalised action messages. The SQLSTATE '02000' handler (no data found) is used as an alternative NOT FOUND handler style.

▶ Code

```
DELIMITER $$
CREATE PROCEDURE CartAbandonmentAnalysis()
BEGIN
    DECLARE v_done      INT DEFAULT 0;
    DECLARE v_uid       INT;
    DECLARE v_uname     VARCHAR(100);
    DECLARE v_cart_val  DECIMAL(10,2);
    DECLARE v_cart_cnt  INT;
    DECLARE v_ord_cnt   INT;
    DECLARE v_action    VARCHAR(200);

    DECLARE CONTINUE HANDLER FOR SQLSTATE '02000' SET v_done = 1;
    DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
        BEGIN
            SET v_cart_val = 0;
            SET v_action   = 'Review manually';
        END;

    DECLARE cur_cart CURSOR FOR
        SELECT u.user_id, u.full_name,
               IFNULL(SUM(ci.quantity * p.price),0),
               COUNT(DISTINCT ci.cart_id),
               (SELECT COUNT(*) FROM orders o2
                WHERE o2.user_id = u.user_id
                  AND o2.status NOT IN ('Cancelled'))
        FROM   users u
        LEFT JOIN cart_items ci ON u.user_id = ci.user_id
        LEFT JOIN products p    ON ci.product_id = p.product_id
        GROUP BY u.user_id, u.full_name;

    DROP TABLE IF EXISTS CartAnalysis;
    CREATE TEMPORARY TABLE CartAnalysis (
        customer      VARCHAR(100),
        cart_items    INT,
        cart_value    DECIMAL(10,2),
        past_orders   INT,
        recommendation VARCHAR(200)
    );

    OPEN cur_cart;
    cart_loop: LOOP
        FETCH cur_cart INTO v_uid, v_uname, v_cart_val, v_cart_cnt, v_ord_cnt;
        IF v_done = 1 THEN LEAVE cart_loop; END IF;

        IF v_cart_cnt > 0 AND v_ord_cnt = 0 THEN
            SET v_action = 'Send welcome discount coupon';
        ELSEIF v_cart_cnt > 0 AND v_cart_val > 10000 THEN
            SET v_action = 'Offer EMI option for high-value cart';
        ELSEIF v_cart_cnt > 0 THEN
            SET v_action = 'Send cart reminder email';
        ELSE
            SET v_action = 'Cart is empty — suggest products';
        END IF;

        INSERT INTO CartAnalysis
        VALUES (v_uname, v_cart_cnt, v_cart_val, v_ord_cnt, v_action);
    END LOOP;
```

```
    CLOSE cur_cart;

    SELECT * FROM CartAnalysis;
END$$
DELIMITER ;

CALL CartAbandonmentAnalysis();
```

▶ Output

```
+------------+------------+------------+-------------+-------------------------------
------+
| customer   | cart_items | cart_value | past_orders | recommendation
|
+------------+------------+------------+-------------+-------------------------------
------+
| Alice Roy  |          1 |     850.00 |           1 | Send cart reminder email
|
| Bob Das    |          1 |   25000.00 |           1 | Offer EMI option for high-value
cart |
| Carol Nair |          1 |   75000.00 |           1 | Offer EMI option for high-value
cart |
+------------+------------+------------+-------------+-------------------------------
------+
3 rows in set (0.04 sec)
```

Conclusion

This assignment helped me understand and apply key MySQL cursor concepts by working on two complete databases. I used declare cursor with complex join queries to retrieve data from multiple related tables. I applied different looping techniques like loop, while, and repeat-until to process records one by one.

I also used if-elseif-else conditions to apply business logic, such as determining booking status, calculating occupancy percentages, categorizing customers, and checking stock levels. To manage cursor execution effectively, I implemented continue handler for not found to exit the loop when no more records are available. Additionally, I used exit handler for sqlexception to handle errors and ensure transactions are rolled back if something goes wrong.

In the AirlineDB, I used procedures for tasks like generating passenger reports, analyzing flight occupancy, and managing refund processes. In the shopsphere database, I focused on customer segmentation, stock alerts, and identifying abandoned carts. These tasks are common in data processing and business intelligence.

One key lesson is that while cursors are useful, they should be used with caution. For large datasets, using set-based sql queries with group by and aggregation functions is much faster and more efficient. Cursors work best in situations where processing each row individually is necessary and cannot be easily achieved with standard sql queries.